

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет о выполнении лабораторной работы №3
по дисциплине «Математическая логика и теория формальных языков»
«Построение контекстно-свободной грамматики подмножества
естественного языка»

Латынь, прошедшее несовершенное время изъявительного наклонения
(Imperfectum indicativi activi)

Студент,
группы 5130201/30101

_____ Мартыненко А. П.

Преподаватель

_____ Востров А. В.

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Описание грамматики Imperfectum indicativi activi	4
1.1 Формы предложений	4
1.1.1 Утвердительные предложения	4
1.1.2 Отрицательные предложения	4
1.1.3 Вопросительные предложения	4
2 Математическое описание	5
2.1 Порождающая грамматика Хомского	5
2.2 Формальное определение грамматики языка	5
2.3 БНФ задаваемого подмножества языка	6
3 Особенности реализации	9
3.1 Структуры данных	9
3.2 Реализация класса генератора предложений	9
3.2.1 Генерация предложения	9
3.2.2 Генерация частей предложения	11
3.3 Реализация класса валидатора предложений	12
3.3.1 Основные методы валидации	12
3.3.2 Валидация частей предложения	14
4 Результаты работы программы	17
Заключение	19
Список использованной литературы	20

Введение

В лабораторной работе необходимо:

1. Построить контекстно-свободную грамматику для латыни в БНФ-нотации, которая позволяет генерировать утвердительные, отрицательные и вопросительные предложения в конкретном времени.
2. Реализовать генератор предложений в соответствии с выбранной грамматикой.
3. Реализовать проверку вводимого предложения на принадлежность грамматике и входному словарю.

Выбранный естественный язык и его подмножество — латынь, прошедшее несовершенное время изъявительного наклонения (Imperfectum indicativi activi)

1 Описание грамматики Imperfectum indicativi activi

Imperfectum indicativi activi (прошедшее время несовершенного вида изъявительного наклонения) выражает длительное или незавершенное действие в прошлом.

Примеры употребления:

- действие в прошлом, которое происходило длительно или было незавершенным
Puer in hortō ambulābat. Мальчик гулял в саду.
- повторяющееся действие в прошлом (привычка)
Quotidiē ad scholam ībāmus. Мы ходили в школу каждый день.
- действия, на фоне которых происходят другие события
Cum epistulam scribarem, amīcus advēnit. Когда я писал письмо, пришёл друг.

Для данного времени сказуемое образуется по схеме
основа настоящего времени + -ba- + окончание

1.1 Формы предложений

1.1.1 Утвердительные предложения

Образуется по схеме: Подлежащее + дополнение + сказуемое, причем к дополнению могут относиться связки прилагательное + существительное.

Примеры:

Caesar librum legēbat. — Цезарь читал книгу.

Caesar librum legēbat cum Rōma valēbat. — Цезарь читал книгу в то время, когда Рим был силен.

1.1.2 Отрицательные предложения

Образуется по схеме: Подлежащее + non + сказуемое + дополнение

Пример: *Ego librum nōn legēbat.* — Цезарь не читал книгу.

1.1.3 Вопросительные предложения

Образуется по схеме: Вопросительное слово + подлежащее + сказуемое + дополнение

Пример: *Cūr tū librum legēbās?* — Почему ты читал книгу?

2 Математическое описание

2.1 Порождающая грамматика Хомского

Порождающая грамматика языка L — это конечный набор правил, позволяющих строить все «правильные» предложения языка L . Применение такой грамматики не дает ни одного «неправильного» предложения, не принадлежащего L .

Порождающая грамматика Хомского: $G = (T, N, S, R)$, где

- T - конечное множество терминалов (слова языка).
- N - конечное множество нетерминалов $T \cap N = \emptyset$.
- $S \in N$ - начальный терминал.
- R - множество правил вывода вида $\alpha \rightarrow \beta$.

Цепочка порождаемого грамматикой языка — это цепочка, состоящая из терминалов, которая получилась в результате произвольного конечного числа подстановок подцепочек в соответствии с правилами грамматики.

Вывод цепочек языка начинается с начального нетерминального символа. Нетерминальные символы используются только для порождения терминальных цепочек.

Языком, порождаемым грамматикой G , называется множество терминальных цепочек, выводимых из начального символа грамматики:

$$L(G) = \{\alpha \in T^* \mid S \Rightarrow_G^* \alpha\}$$

2.2 Формальное определение грамматики языка

Грамматика Imperfectum indicativi activi может быть формально определена как контекстно-свободная грамматика $G = (T, N, S, R)$, где:

- T — множество терминальных символов:
 $T = \textit{pronouns} \cup \textit{proper_nouns} \cup \textit{singular_object} \cup \textit{plural_object} \cup \textit{verb_phrase} \cup \textit{quest_word} \cup \textit{adverb} \cup \textit{conjunction} \cup \textit{adj}$
 - $\textit{pronouns} = \{\textit{is}, \textit{ea}, \textit{id}\}$
 - $\textit{proper_nouns} = \{\textit{Rōma}, \textit{Caesar}, \textit{Cicero}, \textit{Marcus}, \textit{Iūlia}, \textit{Gallia}, \textit{Germania}, \textit{Athēnae}, \textit{Pompēius}, \textit{Augustus}\}$
 - $\textit{singular_object} = \{\textit{librum}, \textit{epistulam}, \textit{domum}, \textit{viam}, \textit{urbem}, \textit{mīlitem}, \textit{equum}, \textit{navem}, \textit{templum}, \textit{hortum}\}$
 - $\textit{plural_object} = \{\textit{librōs}, \textit{epistulās}, \textit{domōs}, \textit{viās}, \textit{urbēs}, \textit{mīlitēs}, \textit{equōs}, \textit{nāvēs}, \textit{templā}, \textit{hortōs}\}$

- verb_phrase = {amābat, monēbat, laudābat, portābat, vocābat, habitābat, spectābat, habēbat, vidēbat, timēbat, debēbat, tacēbat}
- quest_word = {quis, quid, cūr, ubī, quandō, quōmodō, quot, utrum}
- adverb = {semper, saepe, quotidiē, diū, iam, nunc, herī}
- conjunction = {et, sed, aut, enim, igitur, tamen, quod, cum}
- adj = {bonum, bonam, bona, magnam, magnum, magna, parvam, parvum, parva}
- N — множество нетерминальных символов:
 $N = \{\text{sentence, declarative, negative, question, subject, verb_phrase, object, adverbial, pronoun, proper_noun, singular_object, plural_object, quest_word, adverb, conjunction, adj}\}$
- S - начальный символ грамматики: $S = \{\text{sentence}\}$
- R - множество правил вывода (БНФ).

Данная грамматика является контекстно-свободной, то есть относится ко 2 типу по иерархии Хомского, так как:

- Все правила имеют вид $A \rightarrow \alpha$, где $A \in N$ (одионый нетерминал), $\alpha \in (T \cup N)^*$.
- Правые части не зависят от контекста и могут содержать любые комбинации терминалов и нетерминалов.

2.3 БНФ задаваемого подмножества языка

Форма Бэкуса-Наура (БНФ) — форма представления контекстно-свободных формальных грамматик, в которой правые части продукций с одинаковой левой частью объединены.

Правила построения БНФ:

- Нетерминалы помещаются в угловые скобки.
- Символ $::=$ используется вместо стрелки $\rightarrow$.
- Альтернативы одного и того же нетерминала записываются в правой части одного правила через знак $|$.
- A^* означает последовательность из нуля или более выражения A .
- $A?$ означает, что выражение может быть пропущено.

Грамматика подмножества языка в Imperfectum indicativi activi в расширенной форме Бэкуса-Наура:

<sentence> ::= <declarative> "." | <negative> "." | <question> "?"
 <declarative> ::= <subject> <object>? <verb_phrase> <adverbial>*
 <negative> ::= <subject> "non" <verb_phrase> <object>? <adverbial>*
 <question> ::= <quest_word>? <subject> <verb_phrase> <object>? "?"
 <subject> ::= <pronoun> | <proper_noun> | <singular_object>
 <verb_phrase> ::= "amābat" | "monēbat" | "laudābat" | "portābat" |
 "vocābat" | "habitābat" | "spectābat" | "habēbat" | "vidēbat" |
 "timēbat" | "debēbat" | "tacēbat"
 <object> ::= <adj><object> | <object><adj> | <adj><object><adj> |
 <singular_object> | <plural_object>
 <adj> ::= "bonum" | "bonam" | "bona" | "magnam" | "magnum" | "magna"
 | "parvam" | "parvum" | "parva"
 <adverbial> ::= <adverb> | <conjunction> <declarative>
 <pronoun> ::= "is" | "ea" | "id"
 <proper_noun> ::= "Rōma" | "Caesar" | "Cicero" | "Marcus" | "Iūlia"
 | "Gallia" | "Germania" | "Athēnae" | "Pompēius" | "Augustus"
 <singular_object> ::= "librum" | "epistulam" | "domum" | "viam" |
 "urbem" | "mīlitem" | "equum" | "navem" | "templum" | "hortum"
 <plural_object> ::= "librōs" | "epistulās" | "domōs" | "viās"
 | "urbēs" | "mīlitēs" | "equōs" | "nāvēs" | "templa" | "hortōs"
 <quest_word> ::= "quis" | "quid" | "cūr" | "ubī" | "quandō" |
 "quōmodō" | "quot" | "utrum"
 <adverb> ::= "semper" | "saepe" | "quotīdiē" | "diū" | "iam" |
 "nunc" | "herī"
 <conjunction> ::= "et" | "sed" | "aut" | "enim" | "igitur" |
 "tamen" | "quod" | "cum"

Особенности грамматики:

- Допускает необязательные дополнения и обстоятельства для гибкости генерации.
- Допускает сложноподчиненные предложения, разделенные союзом.
- Допускает двустороннюю рекурсию в правиле `<object> ::= <adj><object> | <object><adj> | <singular_object> | <plural_object>`

Примеры предложений:

1. Утвердительное: `Caesar bonum librum legēbat.` (Цезарь читал хорошую книгу.)
2. Отрицательное: `Is epistulam nōn scrībēbat.` (Он не писал письмо.)
3. Вопросительное: `Cūr ea legēbāt?` (Почему она читала?)
4. Утвердительное сложноподчиненное:
`Caesar librum legēbat cum Rōma valēbat.` (Цезарь читал книгу в то время, когда Рим был силен.)

Пример построения дерева предложения представлен на рисунке 1.

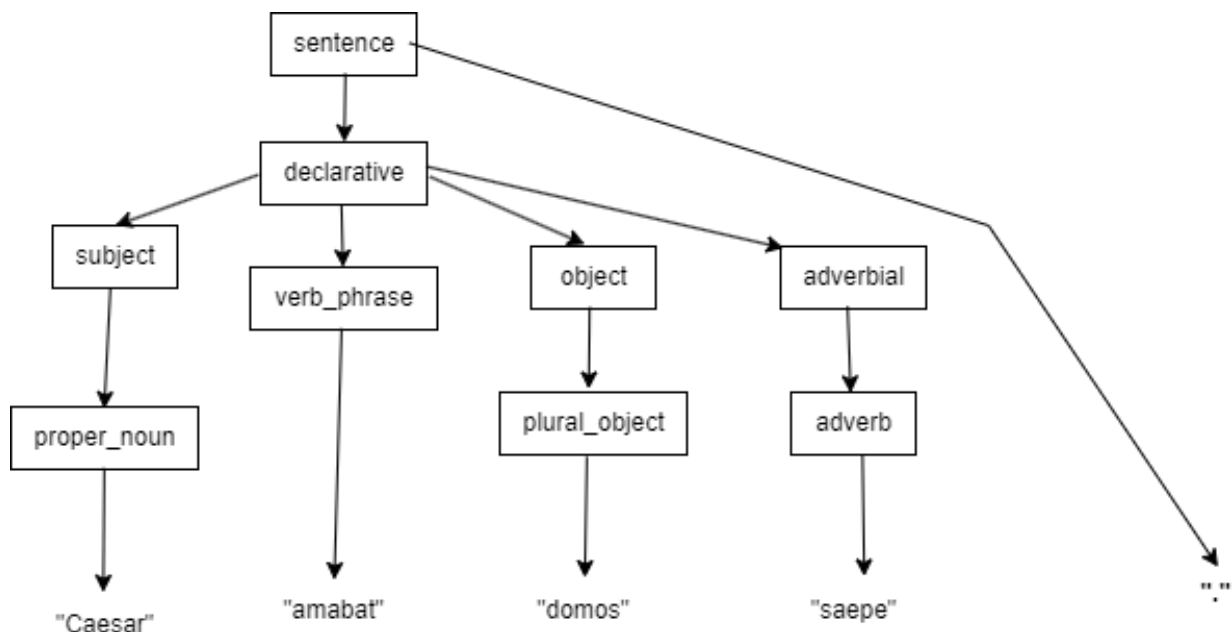


Рис. 1. Дерево предложения

3 Особенности реализации

3.1 Структуры данных

Грамматика представлена следующими структурами данных:

- `pronouns` — список местоимений;
- `proper_nouns` — список собственных имён;
- `singular_objects`, `plural_objects` — списки объектов в единственном и множественном числе;
- `adjectives` — список прилагательных;
- `verb_phrases` — список глагольных фраз;
- `adverbs` — список обстоятельств;
- `conjunctions` — список союзов;
- `quest_words` — список вопросительных слов;
- `all_vocab` — множество всех допустимых слов языка, состоящее из объединения всех списков.

3.2 Реализация класса генератора предложений

3.2.1 Генерация предложения

Метод `generate_sentence` предназначен для реализации генерации одного из трех типов предложения: утвердительного, вопросительного или отрицательного. Предложения генерируются в соответствии с БНФ-грамматикой языка. Метод случайно выбирает тип предложения, вызывает соответствующие функции генерации его частей и добавляет правильную пунктуацию. Первый символ предложения приводится к заглавной букве.

Вход: псевдослучайное число.

Выход: сгенерированное корректное предложение.

Листинг 1. Функция `generate_sentence`

```
1 def generate_sentence(self):
2     ch = random.choice(['declarative', 'negative', 'question'])
3     if ch == 'declarative':
4         core, _ = self.gen_decl_sentence()
5         sentence = f"{core}."
6     elif ch == 'negative':
7         # <negative> ::= <subject> "non" <verb_phrase> <object>? <adverbial>*
8         parts = []
9         parts.append(self.generate_subject())
```

```

10 parts.append("non")
11 parts.append(self.generate_verb_phrase())
12 if random.random() < 0.6:
13 parts.append(self.generate_object())
14 adverb_used = False
15 for _ in range(random.randint(0, 2)):
16 adv, adverb_used = self.generate_adverbial(adverb_used=adverb_used)
17 if adv:
18 parts.append(adv)
19 sentence = " ".join(parts) + "."
20 else:
21 # <question> ::= <quest_word>? <subject> <verb_phrase> <object>? "?"
22 parts = []
23 if random.random() < 0.5:
24 parts.append(random.choice(quest_words))
25 parts.append(self.generate_subject())
26 parts.append(self.generate_verb_phrase())
27 if random.random() < 0.6:
28 parts.append(self.generate_object())
29 sentence = " ".join(parts) + "?"
30 return sentence[0].upper() + sentence[1:]

```

Метод `gen_decl_sentence` предназначен для генерации утвердительного предложения. Утвердительное предложение генерируется в соответствии со структурой: подлежащее + возможное дополнение (с возможными прилагательными) + сказуемое + возможное обстоятельство. Для отслеживания рекурсии используются параметры: `depth` отслеживает текущую глубину вложенности, `max_depth` ограничивает максимальную глубину рекурсии для предотвращения бесконечной генерации. Флаг `adverb_used` отслеживает, было ли уже использовано обстоятельство в текущей цепочке.

Вход: псевдослучайное число, текущая глубина рекурсии, максимальная глубина вложенности, флаг использования обстоятельства.

Выход: сгенерированное корректное утвердительное предложение, обновлённый флаг использования обстоятельства.

Листинг 2. Функция `gen_decl_sentence`

```

1 def gen_decl_sentence(self, depth=0, max_depth=3, adverb_used=False):
2 # <declarative> ::= <subject> <object>? <verb_phrase> <adverbial>*
3 parts = []
4 parts.append(self.generate_subject())
5 if random.random() < 0.6:
6 parts.append(self.generate_object())
7 parts.append(self.generate_verb_phrase())
8 if depth < max_depth:
9 for _ in range(random.randint(0, 1)):
10 adv, adverb_used = self.generate_adverbial(depth=depth + 1, max_depth=
    max_depth, adverb_used=adverb_used)
11 if adv:
12 parts.append(adv)
13 parts = [p for p in parts if p]
14 return " ".join(parts), adverb_used

```

3.2.2 Генерация частей предложения

Метод `generate_adverbial` предназначен для генерации обстоятельства. Обстоятельство может быть представлено либо одиночным наречием, либо придаточным предложением, присоединённым с помощью союза. Таким образом, реализуется возможность построения сложноподчинённых конструкций.

Вход: псевдослучайное число, текущая глубина рекурсии, максимальная глубина вложенности, флаг использования обстоятельства.

Выход: сгенерированное обстоятельство или придаточное предложение, соединённое союзом, обновлённый флаг использования обстоятельства.

Листинг 3. Функция `generate_adverbial`

```
1 def generate_adverbial(self, depth=0, max_depth=3, adverb_used=False):
2 # <adverbial> ::= <adverb> | <conjunction> <declarative>
3 if depth >= max_depth:
4 if adverb_used:
5 return "", adverb_used
6 return random.choice(adverbs), True
7 if random.random() < 0.7:
8 conj_count = random.randint(0, 2)
9 for _ in range(conj_count):
10 conj = random.choice(conjunctions)
11 sub_sent, adverb_used = self.gen_decl_sentence(depth=depth + 1, max_depth=
    max_depth, adverb_used=adverb_used)
12 if sub_sent:
13 return f"{conj} {sub_sent}", adverb_used
14 if not adverb_used:
15 adverb_used = True
16 return random.choice(adverbs), adverb_used
17 return "", adverb_used
```

Метод `generate_subject` предназначен для генерации подлежащего. Подлежащее выбирается случайно из множества местоимений, собственных имён или объектов в единственном числе.

Вход: псевдослучайное число.

Выход: сгенерированное подлежащее.

Листинг 4. Функция `generate_subject`

```
1 def generate_subject(self):
2 # <subject> ::= <pronoun> | <proper_noun> | <singular_object>
3 return random.choice(pronouns + proper_nouns + singular_objects)
```

Метод `generate_verb_phrase` предназначен для генерации сказуемого. Сказуемое выбирается случайно из списка глагольных фраз.

Вход: псевдослучайное число.

Выход: сгенерированное сказуемое.

Листинг 5. Функция `generate_verb_phrase`

```
1 def generate_verb_phrase(self):
2 return random.choice(verb_phrases)
```

Метод `generate_object` предназначен для генерации дополнения. Дополнение может быть представлено как объект (в единственном или множественном числе) или рекурсивная конструкция с добавлением прилагательных до или после объекта.

Вход: псевдослучайное число.

Выход: сгенерированное дополнение.

Листинг 6. Функция `generate_object`

```
1 def generate_object(self, depth=0, max_depth=3):
2 # <object> ::= <adj><object> | <object><adj> | <singular_object> | <
   plural_object>
3 if depth >= max_depth:
4 return random.choice(singular_objects + plural_objects)
5 if random.random() < 0.4:
6 return random.choice(singular_objects + plural_objects)
7 adj = random.choice(adjectives)
8 position = random.choice(['before', 'after'])
9 rest = self.generate_object(depth + 1, max_depth)
10 if position == 'before':
11 return f"{adj} {rest}"
12 else:
13 return f"{rest} {adj}"
```

3.3 Реализация класса валидатора предложений

3.3.1 Основные методы валидации

Метод `validate` предназначен для токенизации входного текста `tokenize`, проверки словаря с помощью функции `check_voc` и синтаксического разбора в зависимости от типа предложения. Также в этом методе формируются детализированные сообщения об ошибках или успехе.

Вход: строка текста.

Выход: булево значение, сообщение об ошибке или успехе.

Листинг 7. Функция `validate`

```
1 def validate(self):
2 if not self.tokenize():
3 return False, self.error
4 if not self.check_voc():
5 return False, self.error
6 self.pos = 0
7 if not self.tokens:
8 return False, "Error! Empty input"
9 last_token = self.tokens[-1]
10 if last_token == '?':
11 if self.match_word(set(quest_words)):
12 pass
13 if not self.parse_subject():
14 return False, "Error! Expected valid subject"
15 if not self.parse_verb_phrase():
16 return False, "Error! Expected valid predicate"
17 saved_pos = self.pos
18 if not self.parse_object():
```

```

19 self.pos = saved_pos
20 if self.pos >= len(self.tokens) or self.tokens[self.pos] != '?':
21     return False, "Error! Extra words at the end of sentence. Expected '?'
    at the end"
22 self.pos += 1
23 elif last_token == '.':
24     if not self.parse_subject():
25         return False, "Error! Expected valid subject"
26     if self.match_word(set(non)):
27         if not self.parse_verb_phrase():
28             return False, "Error! Expected valid predicate"
29     saved_pos = self.pos
30     if not self.parse_object():
31         self.pos = saved_pos
32     else:
33         saved_pos = self.pos
34         if not self.parse_object():
35             self.pos = saved_pos
36             if not self.parse_verb_phrase():
37                 return False, "Error! Expected valid predicate"
38             self.parse_adverbials(max_count=1)
39             if self.pos >= len(self.tokens) or self.tokens[self.pos] != '.':
40                 return False, "Error! Extra words at the end of sentence. Expected '.'
                    at the end"
41             self.pos += 1
42         else:
43             return False, "Error! The sentence should end with '.' or '"
44             if self.pos != len(self.tokens):
45                 return False, "Error! Extra words at the end of sentence"
46             return True, "correct sentence"

```

Метод `tokenize` используется для разбиения строки на токены, проверки корректности заглавной буквы в начале предложения и нормализацию регистра.

Вход: строка текста.

Выход: булевый флаг, показывающий успешность токенизации.

Листинг 8. Функция `tokenize`

```

1 def tokenize(self):
2     text = self.origin_text.strip()
3     if not text:
4         self.error = "Error! Empty message"
5         return False
6     raw_tokens = re.findall(r'\w+|[\.\?]', text)
7     normalized = []
8     for i, token in enumerate(raw_tokens):
9         if token in ['.', '?']:
10             normalized.append(token)
11             continue
12         if i == 0:
13             if not token[0].isupper():
14                 self.error = f"Error! First word '{token}' must start with capital
                    letter"
15             return False
16         if token not in proper_nouns:
17             token = token.lower()
18         else:
19             if token not in proper_nouns:
20                 token = token.lower()

```

```

21     normalized.append(token)
22     self.tokens = normalized
23     return True

```

Метод `check_voc` проверяет, что все слова предложения принадлежат множеству всех допустимых слов языка. Если какого-то слова нет в множестве, записывается ошибка и возвращается булево значение `False`.

Вход: список токенов.

Выход: булевый флаг, показывающий принадлежность всех слов к словарю.

Листинг 9. Функция `check_voc`

```

1     def check_voc(self):
2         for token in self.tokens:
3             if token in ['.', '?']:
4                 continue
5             if token not in all_vocab:
6                 self.error = f"Unknown word: '{token}'"
7             return False
8         return True

```

3.3.2 Валидация частей предложения

Метод `match_word` проверяет принадлежность текущего токена заданному множеству и сдвигает позицию при успехе.

Вход: множество допустимых слов.

Выход: булевый флаг, показывающий принадлежность слова множеству.

Листинг 10. Функция `match_word`

```

1     def match_word(self, cur_vocab):
2         if self.pos >= len(self.tokens):
3             return False
4         if self.tokens[self.pos] in cur_vocab:
5             self.pos += 1
6             return True
7         return False

```

Метод `parse_subject` проверяет корректность подлежащего: токен должен принадлежать одному из трех множеств (местоимения, имена собственные или объекты).

Вход: текущая позиция в токенах.

Выход: булево значение.

Листинг 11. Функция `parse_subject`

```

1     def parse_subject(self):
2         return self.match_word(set(pronouns + proper_nouns + singular_objects))

```

Метод `parse_verb_phrase` проверяет корректность сказуемого: токен должен принадлежать множеству глагольных фраз.

Вход: текущая позиция.

Выход: булево значение.

Листинг 12. Функция `parse_verb_phrase`

```
1 def parse_verb_phrase(self):
2     return self.match_word(set(verb_phrases))
```

Метод `parse_object` реализует рекурсивный разбор дополнения с возможными прилагательными до и после существительного. При этом прилагательные должны быть из соответствующего множества, а существительное - множество объектов в единственном и множественном числе.

Вход: текущая позиция.

Выход: булево значение.

Листинг 13. Функция `parse_object`

```
1 def parse_object(self):
2     saved_pos = self.pos
3     if self.match_word(set(adjectives)):
4         if self.parse_object():
5             return True
6     self.pos = saved_pos
7     if self.match_word(set(singular_objects + plural_objects)):
8         while self.pos < len(self.tokens) and self.match_word(set(adjectives)):
9             pass
10        return True
11    return False
```

Метод `parse_adverbials` проверяет корректность обстоятельств и вложенных придаточных предложений, ограничивая их количество. Придаточным предложением может быть только утвердительное предложение. При этом обстоятельства может не быть в предложении.

Вход: максимальное количество обстоятельств.

Выход: булево значение.

Листинг 14. Функция `parse_adverbials`

```
1 def parse_adverbials(self, max_count=1):
2     count = 0
3     while self.pos < len(self.tokens) and count < max_count:
4         token = self.tokens[self.pos]
5         if token in [',', '?']:
6             break
7         saved_pos = self.pos
8         if self.match_word(set(conjunctions)):
9             if not self.parse_declarative_core():
10                self.pos = saved_pos
11            break
12        count += 1
13        continue
14        if self.match_word(set(adverbs)):
15            count += 1
16            continue
17        self.pos = saved_pos
18        break
19    return True
```

Метод `parse_declarative_core` проверяет базовую структуру утвердительного предложения: подлежащее, возможное дополнение, сказуемое и возможное обстоятельство.

Вход: текущая позиция.

Выход: булево значение.

Листинг 15. Функция `parse_declarative_core`

```
1  def parse_declarative_core(self):
2      if not self.parse_subject():
3          return False
4      saved_pos = self.pos
5      if self.parse_object():
6          pass
7      else:
8          self.pos = saved_pos
9      if not self.parse_verb_phrase():
10         return False
11     self.parse_adverbials()
12     return True
```


4 Результаты работы программы

На рисунках 2- 3 показана проверка введенной строки на соответствие формату латинских предложений.

```
Input latin sentence: Quot domum laudābat bonam epistulās?
Result: correct sentence
Input latin sentence: Urbem non portābat templa sed Rōma domum magnum timēbat.
Result: correct sentence
Input latin sentence: Rōma epistulās bonam bonam laudābat saepe.
Result: correct sentence
Input latin sentence: Domum laudābat?
Result: correct sentence
Input latin sentence: Domum non laudābat bonam epistulās saepe.
Result: correct sentence
```

Рис. 2. Корректно введенные примеры

```
Input latin sentence: Bobum bonam epistulās laudābat.
Result: Unknown word: 'bobum'
Input latin sentence: domum bonam epistulās laudābat saepe.
Result: Error! First word 'domum' must start with capital letter
Input latin sentence: Domum bonam epistulās laudābat saepe saepe.
Result: Error! Extra words at the end of sentence. Expected '.' at the end
Input latin sentence: Domum bonam epistulās laudābat saepe
Result: Error! The sentence should end with '.' or '?'
Input latin sentence: Domum bonam epistulās non laudābat saepe.
Result: Error! Expected valid predicate
Input latin sentence: Quot domum laudābat bonam epistulās saepe?
Result: Error! Extra words at the end of sentence. Expected '?' at the end
Input latin sentence: Quot domum laudābat urbēs parvum sed Rōma domum magnum timēbat?
Result: Error! Extra words at the end of sentence. Expected '?' at the end
Input latin sentence: Quis librum laudābat sed Rōma?
Result: Error! Extra words at the end of sentence. Expected '?' at the end
Input latin sentence: Caesar non librum laudābat.
Result: Error! Expected valid predicate
Input latin sentence:
error, empty string
```

Рис. 3. Некорректно введенные примеры

На рисунке 4 показана генерация рандомной строки, удовлетворяющей формату латинских предложений.

Random sentence: Id non spectābat saepe igitur id viam debēbat. -> [correct]

Random sentence: Marcus non vidēbat iam. -> [correct]

Random sentence: Navem magna epistulās magna magnam portābat. -> [correct]

Random sentence: Rōma amābat? -> [correct]

Random sentence: Is non monēbat librum parvum et domum vocābat diū. -> [correct]

Random sentence: Cicero spectābat? -> [correct]

Random sentence: Librum magna navem magna monēbat. -> [correct]

Random sentence: Equum vocābat quotīdiē. -> [correct]

Рис. 4. Сгенерированные строки

Заключение

В ходе выполнения лабораторной работы была построена контекстно-свободная грамматика подмножества прошедшего несовершенного времени изъявительного наклонения (*Imperfectum indicativi activi*) латыни. На основе этой грамматики была записана БНФ-нотация. На ее основе была разработана программа для проверки корректности введенного предложения на латыни, а также генерации утвердительных, отрицательных и вопросительных предложений, соответствующих данной грамматике. Реализованная программа была протестирована на наборе примеров, включающем как корректные, так и некорректные входные данные.

Достоинства программы:

- генерация предложений осуществляется случайно, также как и выбор конкретных слов из словаря, что обеспечивает разнообразные конструкции;
- при генерации расставлены весовые коэффициенты, которые позволяют добавлять необязательные части предложения в соответствии с приблизительной частотой их появления в процессе использования языка (например, появление дополнения в утвердительном предложении с вероятностью 0.6);
- подключения библиотеки-переводчика, который переводит сгенерированные предложения с латыни на русский;
- можно создавать сложноподчиненные предложения и сложные именные группы с помощью рекурсии.

Недостатки программы:

- при генерации не учитывается смысл слов, из-за чего получаются бессмысленные или стилистически неверные фразы;
- список слов языка фиксирован и не может изменяться динамически.

Масштабирование программы:

- использование логирования вместо выводов ошибок через функцию `print`;
- расширение грамматических структур за счет рассмотрения поэтических стилей.
- возможность генерации текстов заданной длины или структуры;

Список использованной литературы

- [1] Секция «Телематика». — Текст : электронный // tema.spbstu.ru : [сайт]. URL: <https://tema.spbstu.ru/mathem/> (дата обращения: 31.03.2026).
- [2] Ю.Г. Карпов Теория автоматов. - СПб: Издательский дом «Питер», 2003. - 208 с. (дата обращения - 31.03.2026).